Control Flow Structures & Loops

Python Programming

Input

Taking text input from User

```
name = input("Enter your name: ")
print(“Hello ”,name)
```

Output

```
Enter your name: Fawad
Hello  Fawad
```

Taking numeric input from User

```
age = int(input("Enter your age: "))
print("Next year, you will be "+ str(age) + " years old”)
```

Output

```
Enter your age: 25
Next year, you will be 25 years old
```

Conditionals (if statement)

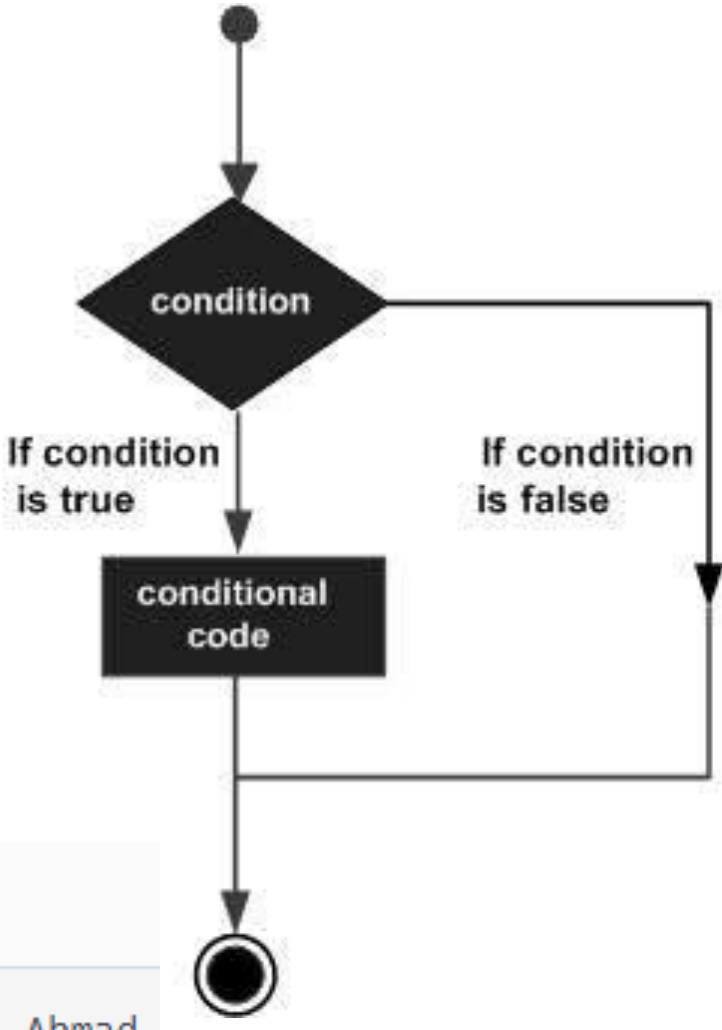
- if <Expression>:
- Statement(s)
- else:
- Statement(s)

- RELATIONAL OPERATORS (Boolean) [0 for false and nonzero value for true in python]
- == 10==10 and 10=10 var==10 and var=10
- !=
- >
- <

Conditionals (Flow diagram) – if statement

Syntax

```
if  <expression>:
  Statement(s)
```



Example

```
name = input("Enter your name: ")
cgpa = float(input("Enter your CGPA: "))
if cgpa>=3.0:
   print(name,'you are PASS')
```

Output

```
Enter your name: Ahmad
Enter your CGPA: 3.5
Ahmad you are PASS
```

Conditionals – if else (Example)

Syntax

if <expression>:

True

else:

False

Example 1

```
name = input("Enter your name: ")
cgpa = float(input("Enter your CGPA: "))
if cgpa>=3.0:
    print(name,'you are PASS')
else:
    print(name,'you are FAIL')
```

Output

```
Enter your name: Ahmad
Enter your CGPA: 2.5
Ahmad you are FAIL
```

Example 2

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
if(age>20):
  print(name,'you are older than 20')
else:
  print('Unfortunately',name,'you are younger than 20')
```

Output

```
Enter your name: Fawad
Enter your age: 20
Unfortunately Fawad you are younger than 20
```

Conditionals (elif)

Syntax

```
if <expression1>:
   statement(s)
elif <expression2>:
   statement(s)
elif <expression3>:
   statement(s)
else:
   statement(s)
```

Conditionals (elif) – Example

Checking multiple expressions for TRUE

Example

```
name = input("Enter your name: ")
cgpa = float(input("Enter your CGPA: "))
if cgpa>=3.0:
  print(name,'your grade is B')
elif cgpa>=3.5:
  print(name,'your grade is B+')
elif cgpa==4.0:
  print(name,'Congratulations, your grade is A')
else:
  print('Unfortunately',name,'you have not qualified')
```

Output

```
Enter your name: Ahmad
Enter your CGPA: 4
Ahmad your grade is B
```

Conditionals (elif) – Example (Modified)

Checking COMBINED multiple expressions using and/or

Example

```
name = input("Enter your name: ")
cgpa = float(input("Enter your CGPA: "))
if cgpa>=3.0 and cgpa<3.5:
  print(name,'your grade is B')
elif cgpa>=3.5 and cgpa<4.0:
  print(name,'your grade is B+')
elif cgpa==4.0:
  print(name,'Congratulations, your grade is A')
elif cgpa<3.0:
  print('Unfortunately',name,'you have not qualified')
else:
  print('You have entered invalid CGPA')
```

Output

```
Enter your name: Fawad
Enter your CGPA: 4
Fawad Congratulations, your grade is A
```

Conditionals (elif) – and/or

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
if age>20 and name==‘Fawad’:
    print(name,'you are older than 20’)
if age>20 and name!='Fawad’:
    print(name,'you are not Fawad’)
elif age==20:
    print("")
else:
    print('Unfortunately',name,'you are younger than 20')
```

Checking COMBINED multiple expressions

Output

```
Enter your name: Fawad
Enter your age: 25
Fawad you are older than 20
Unfortunately Fawad you are younger than 20
```

Control Structures - Nested if

In nested **if** construct:

you can have
an **if...elif...else** construct inside another **if...elif...else** construct.

Syntax

```
if <expression1>:
   statement(s)
   if <expression2>:
      statement(s)
   elif <expression3>:
      statement(s)
   elif <expression4>:
      statement(s)
   else:
      statement(s)
else:
   statement(s)
```

Control Structures - Nested if

```
var = int(input("Please enter a number:"))
if var < 200:
   print ("Expression value is less than 200")
   if var == 150:
      print ("Which is 150")
   elif var == 100:
      print ("Which is 100")
   elif var == 50:
      print ("Which is 50")
   elif var < 50:
      print ("Expression value is less than 50")
else:
   print ("Could not find true expression")

print("Good bye!")
```

If user enters 100

Output

```
Please enter a number:100
Expression value is less than 200
Which is 100
Good bye!
```

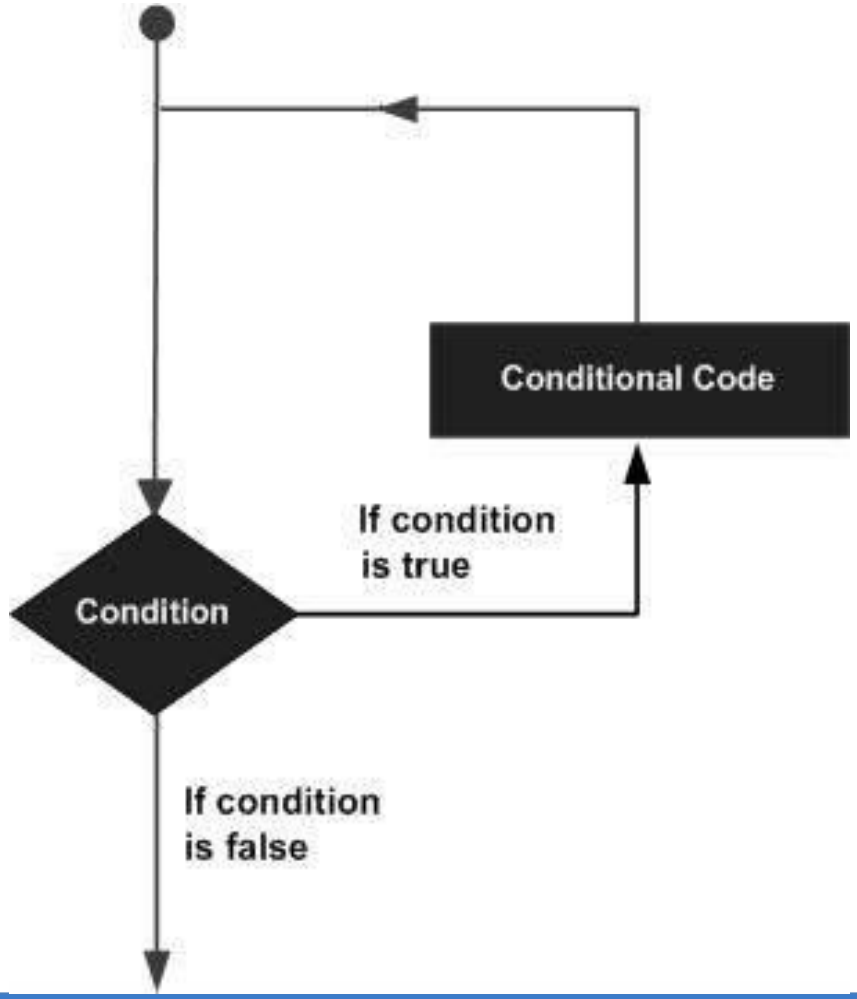
Loops

Python Programming

Loops

- Statements are executed sequentially
- Execute block of code several times

- while loop
- for loop
- Nested loop



```
2
3  num = 9 #a number greater than 2 and less than 10
4  prime = True #a boolean to remember if this number is prime or not
5
6  if num % 2 == 0 and num != 2:
7      print(num,'equals 2 *', num/2)
8      prime = False
9
10 if num % 3 == 0 and num != 3:
11     print(num,'equals 3 *', num/3)
12     prime = False
13
14 if num % 4 == 0 and num != 4:
15     print(num,'equals 4 *', num/4)
16     prime = False
17
18 if num % 5 == 0 and num != 5:
19     print(num,'equals 5 *', num/5)
20     prime = False
21
22 if num % 6 == 0 and num != 6:
23     print(num,'equals 6 *', num/6)
24     prime = False
25
26 if num % 7 == 0 and num != 7:
27     print(num,'equals 7 *', num/7)
28     prime = False
29
30 if num % 8 == 0 and num != 8:
31     print(num,'equals 8 *', num/8)
32     prime = False
33
34 if num % 9 == 0 and num != 9:
35     print(num,'equals 9 *', num/9)
36     prime = False
37
38 if prime:
39     print(num,'is a prime number!')
40 else:
41     print(num,'is not a prime number')
```

for loop

Example

```
for i in range(10):
    print(i)
```

Output

0
1
2
3
4
5
6
7
8
9

Syntax

```
for iterating_var in sequence:
    statements(s)
```

for loop

Example

```
for i in range(1,11):
    print(i)
```

Output

1
2
3
4
5
6
7
8
9
10

Syntax

```
for iterating_var in sequence:
    statements(s)
```

for loop

Example

```
nums = [1, 2, 3, 4, 5]
for num in nums:
        print(num)
```

Output

```
1
2
3
4
5
```

for loop

Example

```
nums = [1, 2, 3, 4, 5]
for num in nums:
    if num==3:
        print('Found')
        break
    print(num)
```

Output

```
1
2
Found
```

Example

```
nums = [1, 2, 3, 4, 5]
for num in nums:
    if num==3:
        print('Found')
        continue
    print(num)
```

Output

```
1
2
Found
4
5
```

Nested for loop

Example

```
nums = [1, 2, 3, 4, 5]
letters = ['a','b','c']
for num in nums:
    for letter in letters:
        print(num,letter)
```

Output

```
1 a
1 b
1 c
2 a
2 b
2 c
3 a
3 b
3 c
4 a
4 b
4 c
5 a
5 b
5 c
```

Nested for loop

Example

```
nums = [1, 2, 3, 4, 5]
letters = ['abc']
for num in nums:
    for letter in letters:
        print(num,letter)
```

Output

```
1 abc
2 abc
3 abc
4 abc
5 abc
```

Nested for loop (Home task)

Output

Example

```
nums = [1, 2, 3, 4, 5]
for num in nums:
    for letter in 'abc':
        print(num,letter)
```

Take a number form user and check:

If the number is Prime?

If the number is odd?

If the number is even?

While loop

Example

```
x=0
while x<10:
    print(x)
    x+=1
```

Output

```
0
1
2
3
4
5
6
7
8
9
```

While loop

Example

```
x=0
while x<10:
    if x==5:
        break
    print(x)
    x+=1
```

Output

```
0
1
2
3
4
```

While loop

Example

```
x=0
while x<10: while True:
    if x==5:
        break
    print(x)
    x+=1
```

Output

0

1

2

3

4

Collections

- Multiple different values stored in a single variable
- Work with them collectively

- X = 16 #int
- Y= “Fawad” #string
- Z= 4.7 #float

Implicitly assigned by Python

- Another type of built-in data type

Collections

- List
  - General purpose
  - Most widely used
  - Grow/shrink size as req.
  - Sequence type
  - Sortable

```
fruits = ["Apple", "Mango","Banana"]
```

- Tuple
  - Immutable(can't add/change)
  - Useful for fixed data
  - Faster than lists
  - Sequence type

```
days = ("Monday", "Tuesday", "Wednesday")
```

- Dict
  - Key/value pairs
  - Unordered

```
student = {
"name": "Fawad",
"age": 25,
"cgpa": 3.5}
```

  - Operations include indexing, slicing, adding, multiplying, and checking for membership can be performed.

Sequences (String, List, tuple)

- Indexing
- Slicing
- Adding/concatenating
- Multiplying
- Checking membership
- Iterating
- len
- max
- min

- x[6]
- x[1:4]
- +
- *
- in/not in
- for i in x:

Built in functions (String)

- isalnum()

Example

var="lengthofstring17"

print(var.isalnum())

Output

True

- len(var1)
- Returns length of string

Example

str = "Length of string:"

print("Length of string:",len(str))

Output

Length of string: 17

Built in functions (String)

- count()

count(sub, start= 0,end)

Example

```
str = "Length of string is:"
sub="s"
print("Substring count is",str.count(sub,4,40))
```

Output

Substring count is 2

Built in functions (String)

- center()

center(width, fillchar)

Example

```
str = "Length of string is:"
print ("Character filled string is: ", str.center(24, 'a'))
```

Output

```
Character filled string is:  aaLength of string is:aa
```

- **indexing**
  - Access any item in the sequence using its index

String

```
x = 'frog'
print (x[3])                    # prints 'g'
```

List

```
x = ['pig', 'cow', 'horse']
print (x[1])                    # prints 'cow'
```

- **slicing**
  - Slice out substrings, sublists, subtuples using indexes

[start : end+1 : step]

```
x = 'computer'
```

Code	Result	Explanation
`x[1:4]`	'omp'	Items 1 to 3
`x[1:6:2]`	'opt'	Items 1, 3, 5
`x[3:]`	'puter'	Items 3 to end
`x[:5]`	'compu'	Items 0 to 4
`x[-1]`	'r'	Last item
`x[-3:]`	'ter'	Last 3 items
`x[:-2]`	'comput'	All except last 2 items

- **adding / concatenating**
  - Combine 2 sequences of the same type using **+**

String

```
x = 'horse' + 'shoe'
print (x)                    # prints 'horseshoe'
```

List

```
x = ['pig', 'cow'] + ['horse']
print (x)                    # prints ['pig', 'cow', 'horse']
```

- **multiplying**
  - Multiply a sequence using *

String

```
x = 'bug' * 3
print (x)          # prints 'bugbugbug'
```

List

```
x = [8, 5] * 3
print (x)          # prints [8, 5, 8, 5, 8, 5]
```

- **checking membership**
  - Test whether an item is **in** or **not in** a sequence

String

```
x = 'bug'
print ('u' in x)                    # prints True
```

List

```
x = ['pig', 'cow', 'horse']
print ('cow' not in x)              # prints False
```

- **iterating**
  - Iterate through the items in a sequence

Item

```
x = [7, 8, 3]
for item in x:
    print (item * 2)                    # prints 14, 16, 6
```

- **number of items**
  - Count the number of items in a sequence

String

```
x = 'bug'
print (len(x))                    # prints 3
```

List

```
x = ['pig', 'cow', 'horse']
print (len(x))                    # prints 3
```

- **minimum**
  - Find the minimum item in a sequence lexicographically
  - alpha or numeric types, but cannot mix types

String

```
x = 'bug'
print (min(x))                    # prints 'b'
```

List

```
x = ['pig', 'cow', 'horse']
print (min(x))                    # prints 'cow'
```

- **maximum**
  - Find the maximum item in a sequence
  - alpha or numeric types, but cannot mix types

String

```
x = 'bug'
print (max(x))                    # prints 'u'
```

List

```
x = ['pig', 'cow', 'horse']
print (max(x))                    # prints 'pig'
```

- **sum**
  - Find the sum of items in a sequence
  - entire sequence must be numeric type

String -> Error

```
x = [5, 7, 'bug']
print (sum(x))                    # error!
```

List

```
x = [2, 5, 8, 12]
print (sum(x))                    # prints 27
print (sum(x[-2:]))               # prints 20
```

- **sorting**
  - Returns a new list of items in **sorted** order
  - Does not change the original list

String

```
x = 'bug'
print (sorted(x))          # prints ['b', 'g', 'u']
```

List

```
x = ['pig', 'cow', 'horse']
print (sorted(x))          # prints ['cow', 'horse', 'pig']
```

- **count (item)**
  - Returns count of an item

String

```
x = 'hippo'
print (x.count('p'))                    # prints 2
```

List

```
x = ['pig', 'cow', 'horse', 'cow']
print (x.count('cow'))                  # prints 2
```

- **index (item)**
  - Returns the index of the first occurrence of an item

String

```
x = 'hippo'
print (x.index('p'))                    # prints 2
```

List

```
x = ['pig', 'cow', 'horse', 'cow']
print (x.index('cow'))                  # prints 1
```

- **unpacking**
  - Unpack the n items of a sequence into n variables

```
x = ['pig', 'cow', 'horse']
a, b, c = x                    # now a is 'pig'
                               # b is 'cow',
                               # c is 'horse'
```

Note:
The number of variables must exactly match the length of the list.

- **delete**
  - Delete a list or an item from a list

```
x = [5, 3, 8, 6]
del(x[1])                  # [5, 8, 6]

del(x)                     # deletes list x
```

- **append**
  - Append an item to a list

```
x = [5, 3, 8, 6]
x.append(7)                 # [5, 3, 8, 6, 7]
```

- **insert**
  - Insert an item at given index `x.insert(index, item)`

```
x = [5, 3, 8, 6]
x.insert(1, 7)              # [5, 7, 3, 8, 6]

x.insert(1,['a','m'])   # [5, ['a', 'm'], 7, 3, 8, 6]
```

- **extend**
  - Append an sequence to a list

```
x = [5, 3, 8, 6]
y = [12, 13]
x.extend(y)                # [5, 3, 8, 6, 7, 12, 13]
```

- **pop**
  - Pops last item off the list, and returns item

```
x = [5, 3, 8, 6]
x.pop()              # [5, 3, 8]
                     # and returns the 6

print(x.pop())       # prints 8
                     # x is now [5, 3]
```

- **remove**
  - Remove first instance of an item

```
x = [5, 3, 8, 6, 3]
x.remove(3)                    # [5, 8, 6, 3]
```

- **reverse**
  - Reverse the order of the list

```
x = [5, 3, 8, 6]
x.reverse()                    # [6, 8, 3, 5]
```

- **sort**
  - Sort the list in place

```
x = [5, 3, 8, 6]
x.sort()                    # [3, 5, 6, 8]
```

Note:

sorted(x) returns a *new* sorted list without changing the original list x.

x.sort() puts the items of x in sorted order (sorts in place).